COACHEASILY

SalesOS

Sales CRM Architecture & Implementation Blueprint

A practical engineering blueprint for lead management, call records, follow-up automation, role-based access and sales analytics.

PROPOSED ARCHITECTURE • OCTOBER 2026

Prepared for product, technology and sales leadership

Document guide

This blueprint describes the proposed architecture and phased build plan. It is not a claim that the platform has already been implemented or tested.

Section	Contents
1	Executive summary and business outcomes
2	Product workflow and modules
3	Technology stack and system architecture
4	Roles, permissions and access boundaries
5	Database schema and critical constraints
6	REST API catalogue
7	Follow-up automation and reliability
8	Dashboards, KPIs and reporting
9	Personal Android calling feasibility
10	AI Sales Copilot and safeguards
11	Security and production readiness
12	Phase-by-phase implementation prompts
13	Roadmap and launch checklist

1. Executive summary

CoachEasily SalesOS is a new, independent CRM intended to help the sales team work every lead consistently: assign ownership, capture call outcomes, preserve client context, schedule follow-ups, escalate overdue tasks and measure conversions. The CEO receives organization-wide visibility; each manager sees only their permitted team scope; each salesperson works their authorized leads.

Primary success principle: every active lead must have a responsible owner and a clear next action, or appear in a visible exception queue for resolution.

Important calling constraint: salespeople currently use personal Android phones for normal cellular calls. A web CRM cannot reliably capture every such call or record conversations by itself. Manual call logging must always work; device-level call metadata or recording is a separately validated, permission- and policy-dependent enhancement.

Business outcomes

- Reduce missed leads through task ownership, due dates, reminders and escalation.
- Keep a chronological client history of calls, notes, objections, interest and next steps.
- Give managers a reliable view of team workload and follow-up compliance.
- Give the CEO auditable organization-wide sales and revenue analytics.
- Add AI assistance only where source data and privacy permissions support it.

2. Product workflow and modules

Lead lifecycle

Step	System behaviour
1. Lead intake	Create or import a lead from approved sources; validate fields and flag possible duplicates.
2. Assignment	Assign a responsible salesperson manually or through configurable round-robin rules.
3. Call activity	Open the phone dialer and then capture outcome, notes, objections and interest.
4. Next action	Create a callback, demo, email or follow-up task with due date and owner.
5. Reminder engine	Notify the salesperson, retry failed deliveries and escalate overdue work according to rules.
6. Manager review	Show unassigned leads, overdue tasks, high-priority leads and unresolved exceptions.
7. Deal outcome	Record Won/Lost, close date, value and reason; preserve the full history.

Core modules

- CEO dashboard, manager dashboard and salesperson workspace.
- Lead management, source attribution, duplicate review, assignment history and pipeline.
- Call logs, timestamped notes, activity timeline and optional recording references.
- Tasks, follow-up calendar, reminders, overdue queues and manager escalation.
- Deals, targets, conversion analytics, exports and audit logs.
- AI Sales Copilot and optional Android companion integration after feasibility testing.

3. Technology stack and architecture

Layer	Recommended choice	Responsibilities
Web UI	Next.js App Router, TypeScript, Tailwind CSS, shadcn/ui, Framer Motion	Responsive CEO, manager and sales panels.
API	NestJS / Node.js	REST APIs, business rules, authorization and validation.
Database	PostgreSQL + Prisma	Transactional source of truth and versioned migrations.
Background jobs	Redis + BullMQ	Due reminders, retries, scheduled jobs and escalations.
Live updates	SSE initially	Dashboard updates and notification events.
Authentication	Secure server-managed sessions	Login, logout, session revocation and account security.
File storage	Cloudflare R2 or S3-compatible storage	Protected attachments and approved recording assets.
AI	OpenAI or Gemini through server-side abstraction	Summaries, next-step suggestions and coaching.
Delivery	Docker + Nginx or managed cloud	Deployment, health checks, monitoring and recovery.

Architecture approach

Start with a modular monolith: one structured NestJS API, one PostgreSQL database and a separate background worker process. This keeps deployment and debugging simpler while preserving clear modules. Split services only when measurable scale or ownership needs justify it.

Suggested repository structure

```
coacheasily-sales-crm/
  apps/
    web/                # Next.js dashboard
    api/                # NestJS REST API
    worker/             # BullMQ jobs
    mobile-companion/   # optional later
  packages/
    shared-types/
    validation/
    ui/
  prisma/
    schema.prisma
    migrations/
  docs/
    architecture.md
    api-spec.md
    security.md
  docker-compose.yml
  README.md
```

4. Roles, permissions and access boundaries

Capability	CEO	Sales Manager	Salesperson
Organization-wide analytics	All	No	No
Team performance	All teams	Own permitted teams	Own performance
Lead visibility	All authorized leads	Team-scoped leads	Assigned/authorized leads
Lead reassignment	Organization-wide	Within permitted scope	No by default
Call records and notes	All authorized records	Team scope	Own authorized records
Deals and revenue	All	Team scope	Own deals
Users and roles	Authorized admin controls	No	No
Audit history	Organization-wide	Team-scoped view	Own activity where allowed
Organization settings	Yes	No	No

Security rules

- Authorization is enforced on the server for every request; hiding UI controls is not security.
- Do not trust browser-supplied organization IDs, manager IDs, roles or assignment owners.
- Use deny-by-default policies and test cross-team and cross-organization access.
- Keep privileged actions auditable, including CEO actions and data exports.
- Use expiring signed links for recordings and attachments; validate scope before issuing them.
- When an employee is deactivated, route open leads to a reassignment queue rather than orphaning them.

5. Database schema

Use UUID primary keys, foreign keys, created/updated timestamps, organization scope where applicable, and transactions for related business updates. Store timestamps in UTC (TIMESTAMPTZ); render them in the user's configured timezone.

Table	Key fields / purpose
organizations	id, name, settings, created_at
users	id, organization_id, name, email, password_hash, role, manager_id, is_active

Table	Key fields / purpose
teams	id, organization_id, name
team_memberships	team_id, user_id, membership_role, created_at
leads	id, organization_id, full_name, phone_e164, email, source_id, assigned_to, status, interest_level, next_action_at, next_action_type, estimated_value, lost_reason
lead_assignments	id, lead_id, from_user_id, to_user_id, changed_by, reason, changed_at
lead_status_history	id, lead_id, old_status, new_status, changed_by, reason, created_at
lead_notes	id, lead_id, author_id, body, visibility, created_at
call_logs	id, lead_id, user_id, source, direction, outcome, provider_call_id, started_at, duration_seconds, notes, created_at
call_notes	id, call_id, author_id, summary, objections, next_steps, created_at
tasks	id, lead_id, assigned_to, created_by, type, due_at, status, priority, completed_at, completion_note
deals	id, lead_id, owner_id, stage, amount, currency, won_at, lost_at, lost_reason
recording_assets	id, call_id, storage_key, consent_status, retention_until, created_at
notification_deliveries	id, user_id, task_id, channel, status, attempts, provider_reference
audit_logs	id, organization_id, actor_id, action, entity_type, entity_id, metadata, created_at

Critical constraints and indexes

- Index lead lists by (organization_id, assigned_to, status, created_at).
- Index due tasks by (status, due_at, assigned_to); index call history by (lead_id, created_at DESC).
- Normalize phone numbers to E.164 where possible; flag potential duplicates for review instead of silently merging.
- Use import idempotency keys to prevent duplicate rows on retry.
- Use transactions for call + task + status-history writes and for deal closure changes.
- Use a transactional outbox or equivalent reliable event pattern to avoid losing reminder events between database commits and queue publishing.

6. REST API catalogue

Base path: /api/v1. All endpoints require authentication unless explicitly designated otherwise. Apply server-side validation, authorization, pagination, rate limiting and consistent error responses.

Area	Method and endpoint	Purpose
Auth	POST /auth/login	Authenticate user
Auth	POST /auth/logout	Revoke session
Auth	GET /auth/me	Current user and effective permissions
Users	GET, POST /users	List/create authorized users
Users	PATCH /users/:id	Update permitted user fields
Teams	GET, POST /teams	List/create teams
Leads	GET, POST /leads	Search/list and create leads
Leads	GET, PATCH /leads/:id	Read/update lead
Leads	POST /leads/import	Validated CSV import
Leads	POST /leads/:id/assign	Assign or reassign lead

Area	Method and endpoint	Purpose
Leads	POST /leads/:id/status	Change pipeline stage
Leads	GET /leads/:id/timeline	Calls, notes, tasks and status history
Calls	POST /leads/:id/calls	Create call outcome record
Calls	GET /leads/:id/calls	Read permitted call history
Tasks	POST /leads/:id/tasks	Schedule follow-up
Tasks	GET /tasks	My tasks or authorized team tasks
Tasks	POST /tasks/:id/complete	Complete a task
Deals	GET /deals; POST /leads/:id/deals	List/create deals
Deals	POST /deals/:id/close	Close Won/Lost with validation
Dashboards	GET /dashboard/ceo	Organization-level analytics
Dashboards	GET /dashboard/manager	Team analytics
Dashboards	GET /dashboard/salesperson	Personal performance
Reports	GET /reports/follow-ups	Due and overdue reporting
Reports	GET /reports/conversions	Conversion funnel
Devices	POST /devices/register	Register approved companion device
Telephony	POST /integrations/telephony/webhook	Validate provider events
Audit	GET /audit-logs	Authorized audit history

API design requirements

- Use stable response envelopes and standard status codes; never return secrets or password hashes.
- Use cursor or page-based pagination with bounded page sizes.
- Use idempotency keys for imports, webhooks and retryable create operations.
- Verify webhook signatures and reject replayed or invalid events.
- Document endpoints with OpenAPI and provide example requests and responses.

7. Follow-up automation and reliability

Outcome / event	Expected system action
NO_ANSWER	Create retry task using configurable delay and working-hours rules.
BUSY	Offer callback scheduling and prioritize the client's requested time.
SWITCHED_OFF	Schedule a delayed retry; route repeated failures for manager review.
CONNECTED	Capture notes, interest, objections and a next action.
CALLBACK	Record preferred date/time and create a high-visibility task.
OVERDUE	Notify owner; escalate to manager using configurable thresholds.
WON	Record amount and close date; resolve sales follow-ups and create onboarding tasks if needed.
LOST	Require reason, preserve history and apply approved re-engagement policy.
OWNER DEACTIVATED	Create reassignment exception for every open lead.

Reliability design

- BullMQ jobs need bounded retries, exponential backoff where appropriate and dead-letter handling.
- Use idempotent notification keys to prevent duplicate reminders.
- Use task locking or atomic state transitions when multiple workers may process the same job.
- Track delivery status and provider error codes; do not treat queued as delivered.
- Make reminder schedules configurable by organization, team and working hours.
- Do not automatically mark a lead Lost only because a client did not answer.

8. Dashboards and KPI definitions

Metric	Suggested definition / guardrail
New leads	Count of leads created in the selected date range.
Call attempts	Count of call records; report separately from unique clients.
Connected calls	Calls marked connected, based on recorded disposition or verified provider event.
Follow-up compliance	Completed due tasks on time divided by eligible tasks due in the period.
Overdue tasks	Open tasks whose due_at is earlier than now; exclude cancelled/completed tasks.
Lead conversion	Won leads divided by the defined lead cohort; disclose date range and denominator.
Revenue	Sum of won deal amounts using a consistent currency and close-date rule.
Unassigned leads	Active leads with no current owner.
No-next-action leads	Active leads without a scheduled next action; show as an exception metric.

CEO sees all authorized organization metrics; managers see their permitted team scope; salespeople see their personal activity and target progress. Dashboard filters must not bypass server-side authorization.

9. Personal Android calling feasibility

MVP approach — reliable first

- Use a tel: link to open the phone dialer from the lead detail page.
- After the call, require a clear disposition form: connected, no answer, busy, switched off, callback, failed, won or lost.
- Capture notes, interest level, objections and next action; update the lead timeline.
- Keep reminders and CRM activity independent of device call-log permissions.

Optional Android companion — feasibility gate

- Test representative Android versions and devices before promising automatic call detection.
- Evaluate supported APIs, permissions, background restrictions and Google Play policy.
- Use explicit disclosure and user-controlled permissions; do not bypass OS security or silently record.
- Distinguish manual records from device/provider-verified events.
- Only consider recording after a separate technical, consent, privacy and legal review.

Important limitation: Android call-screening and call-log capabilities are restricted and vary by use case, device and policy. A normal website cannot guarantee unrestricted access to personal cellular calls. Recording and transcription are not MVP guarantees.

10. AI Sales Copilot

Potential features

- Summarize salesperson-entered notes or lawfully obtained, consented transcripts.
- Extract requirements, objections, interest, budget and proposed next steps.
- Draft follow-up messages for salesperson review.
- Prepare a call brief from authorized CRM history.
- Recommend lead priority using explainable rules and available data.
- Track AI usage, latency, failures and cost.

Guardrails

- AI output is a suggestion, not a verified transcript or factual record unless source data supports it.
- Never fabricate call content, close deals automatically or send external messages without approval.
- Treat client text as untrusted input; protect against prompt injection.
- Minimize personal data sent to providers and apply access controls to prompts and outputs.
- Keep core CRM workflows usable if AI is unavailable.

11. Security and production readiness

- Secure session cookies, CSRF protections where applicable, password hashing and session revocation.
- Role and organization checks on every API and export; negative authorization tests.
- Request validation, rate limits, upload validation and safe error responses.
- Secrets only in environment/secret manager; never commit production keys.
- Expiring signed storage URLs, encryption in transit and appropriate at-rest protection.
- Audit privileged actions, ownership changes, status changes, exports and recording access.
- Document retention, deletion, consent, export and incident response policies.
- Backups with a tested restore procedure, monitoring, structured logs and health checks.
- Database migration plan, rollback/recovery strategy and performance tests.
- Accessible responsive UI with loading, empty, error, success and permission-denied states.

12. Phase-by-phase implementation plan

Phase 0 — Master instructions

Create CLAUDE.md or equivalent project instructions. Set stack, business rules, security standards, testing gates and documentation requirements. Inspect the repository before changing files.

Phase 1 — Foundation and RBAC

Set up Next.js, NestJS, PostgreSQL, Prisma, Redis/BullMQ and local development. Build auth, organizations, teams, memberships, roles, secure sessions and deny-by-default authorization. Test cross-team and cross-organization access.

Phase 2 — Lead management

Build leads, sources, assignments, status history, notes, search, filters, pagination, CSV import, duplicate review, ownership history and lead timeline. Ensure imports are idempotent and all transitions are auditable.

Phase 3 — Calls, tasks and follow-ups

Build call dispositions, notes, tasks, calendar, reminders, retries and overdue escalation. Use transactions and idempotent jobs. Manual call logging must work regardless of Android permissions.

Phase 4 — Analytics

Build CEO, manager and salesperson dashboards, KPI definitions, team/source filters, exports and reconciliation tests. Enforce server-side scope on every metric.

Phase 5 — AI Sales Copilot

Add a provider abstraction, summaries, objection extraction, call preparation and suggested next steps. Keep keys server-side; allow review/edit; protect privacy and avoid fabricated content.

Phase 6 — Android feasibility

Run a device-level proof of concept for personal cellular call metadata. Review permissions, background restrictions, Play policies and consent. Build a companion only where supported; keep manual logging as fallback.

Phase 7 — Production launch

Complete security review, monitoring, backup restore test, job recovery, data retention, performance tests, accessibility review, operational runbook and deployment approval.

Reusable acceptance gate for every phase

- Inspect existing files and dependencies before modifying them.
- Implement only the agreed phase scope; do not silently skip requirements.
- Run lint, type checks, unit tests, integration tests and production build.
- Fix errors rather than suppressing them.
- Update setup docs, migrations, environment variables, API documentation and known limitations.
- Report changed files and real test results; never claim unrun tests passed.

13. Launch checklist

Area	Acceptance criteria
Identity & permissions	Role tests pass; cross-team and cross-organization access is denied.
Lead workflow	Imports, duplicate review, assignment and complete history work.
Call records	Manual disposition, notes and lead timeline work reliably.
Follow-ups	Tasks, reminders, retries and escalation survive worker restarts.
Analytics	KPIs reconcile with database records and obey role scope.
AI	Suggestions are reviewable, provider failures do not break CRM.
Android	Compatibility and policy limitations are documented; no unsupported guarantees.
Production	Security review, monitoring, backup restore and runbook are complete.

Recommended build order

Complete Phases 0–3 first. This creates a useful CRM with lead ownership, call records and follow-up accountability. Add analytics next, then AI and Android feasibility work. The product's core success measure is reliable lead follow-up—not automatic call recording.

Document status: Proposed architecture. All technical choices and implementation details should be validated against the actual repository, deployment environment, current Android policy and privacy requirements before production launch.